Theme — Merchant Integration & API Blueprint

Audience: Backend Engineering (Sameeha, Gauthami)
Status: v1.0 — Implementation Reference
Owner: Orbit-360 Platform Team

---

1. Project Overview

1.1 Goal

Deliver a production-grade integration layer that connects the **Next.js Electronics Storefront** to the **Orbit-360 multi-tenant API** at `api.evoclabs.com`. The storefront must render merchant-specific catalog, branding, and policies in real time while supporting a **frictionless purchase path** from product impression to order confirmation.

1.2 Core Philosophy — Direct-to-Consumer (DTC)

- **No customer auth.** No login, no registration, no password recovery flows on the storefront.
- **Guest-only checkout.** Identity is captured exclusively through the delivery form (name, phone, address, pincode).
- **Single-tap intent.** "Buy Now" is the primary CTA; cart/wishlist are secondary surfaces.
- **Merchant-scoped isolation.** Every read and write is bound to the active merchant tenant — no cross-store leakage is acceptable.

1.3 Non-Goals (out of scope for this phase)

- Customer accounts, saved addresses, loyalty programs.
- Subscription products, recurring billing.
- B2B/wholesale pricing tiers.

---

2. Team Task Division

2.1 Sameeha — Catalog & Data Layer

Owns everything the storefront *reads* to render merchandise and brand identity.

Domain	Endpoints (indicative)	Responsibilities
Products	`GET /api/products`, `GET /api/products/:slug`	List, filter, paginate, single-product fetch with variants & stock.
Categories	`GET /api/categories`, `GET /api/categories/:slug`	Tree hydration, breadcrumbs, category-product joins.
Store Branding	`GET /api/store/settings`	Logo, colors, typography, hero banners, footer config.
Merchant Settings	`GET /api/store/policies`, `GET /api/store/contact`	Shipping/return policy text, support contact, social handles.
SEO Metadata	`GET /api/products/:slug/meta`	Title, description, canonical URL, OG tags.

Deliverable: A typed `catalog-client` SDK exposing strongly-typed read APIs consumable from Next.js Server Components.

2.2 Gauthami — Transaction & Order Lifecycle

Owns everything that *mutates* state or touches money/logistics.

Domain	Endpoints (indicative)	Responsibilities
Pincode Serviceability	`GET /api/logistics/pincode/:code`	Validate deliverability, surface ETA, COD eligibility.
Direct Order	`POST /api/orders/direct`	Single-call guest checkout (product + buyer + address + payment intent).
Order Tracking	`GET /api/orders/track/:trackingId`	Public, tokenized, no-auth tracking.
Order Status Webhook	`POST /api/webhooks/order-status`	Inbound status updates from logistics partners.

Deliverable: A `commerce-client` SDK + a public `/track/:id` endpoint that is rate-limited and tokenized.

---

3. Technical Specifications

3.1 Base URL

```
Production:  https://api.evoclabs.com
Staging:     https://staging.api.evoclabs.com
```

All requests **must** be HTTPS. Plaintext is rejected at the edge.

3.2 Tenant Identification

Every request **must** identify the merchant tenant. Two mechanisms are supported, in priority order:

Option A — Header (preferred for SSR / API routes):

```
GET /api/products HTTP/1.1
Host: api.evoclabs.com
x-store-id: <merchant_uuid>
Content-Type: application/json
```

Option B — Subdomain (used when storefront is rendered on `<merchant>.evoclabs.shop`):
The API's tenant resolver inspects the `Origin` / `Host` header and maps subdomain → `storeId` server-side. No client work required beyond setting `Origin` correctly.

> **Rule:** If both are present, `x-store-id` wins. If neither resolves to a valid tenant, the API returns `400 TENANT_REQUIRED`.

3.3 Data Models (reference)

Authoritative definitions live in `backend/prisma/schema.prisma`. Storefront types **must** be derived from these — do not redefine.

```
model Product       { … }   // schema.prisma:296
model Order         { … }   // schema.prisma:341
model StoreSettings { … }   // schema.prisma:382
```

Type generation: Run `prisma generate` in `backend/` and re-export the relevant DTOs through the SDK packages — never expose raw Prisma types to the Next.js client bundle.

3.4 Standard Response Envelope

```
{
  "ok": true,
  "data": { … },
  "meta": { "requestId": "req_…", "tookMs": 42 }
}
```

On error:

```
{
  "ok": false,
  "error": { "code": "PINCODE_NOT_SERVICEABLE", "message": "…" },
  "meta": { "requestId": "req_…" }
}
```

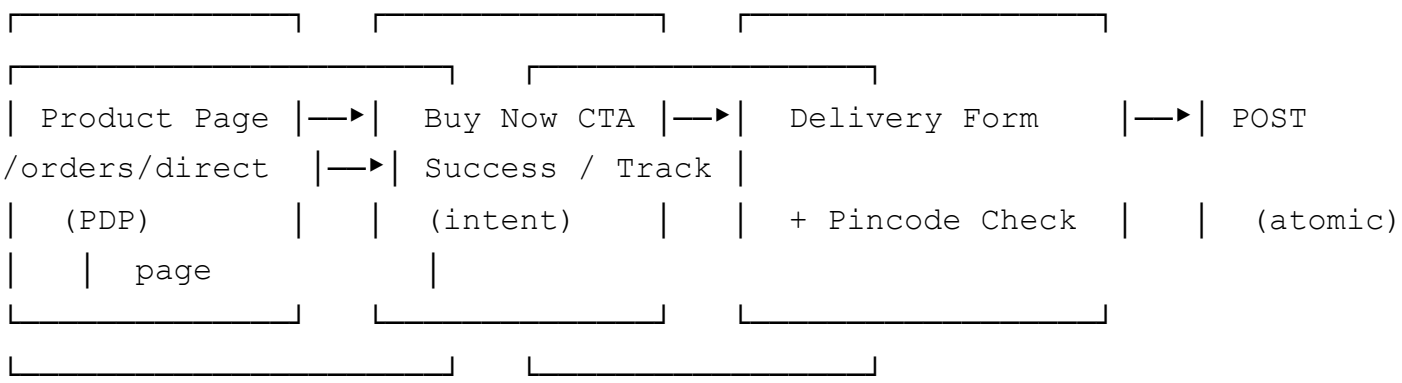
Both SDKs must throw a typed `OrbitApiError` carrying `code`, `message`, and `requestId`.

---

4. The "Direct Checkout" Workflow

End-to-end purchase path. Reference implementation lives behind `POST /api/orders/direct`.

4.1 Sequence



4.2 Step-by-step Contract

Step 1 — PDP load (Sameeha)

```
GET /api/products/:slug
x-store-id: <merchant_uuid>
```

Returns product + variants + stock + SEO meta. Hydrate Next.js `generateMetadata()` from the `meta` block.

Step 2 — Buy Now click

Pure client-side state transition. Selected variant + quantity are stashed in a checkout draft (memory only — no localStorage of buyer PII).

Step 3 — Delivery form: Pincode validation (Gauthami)

Triggered on `onBlur` of the pincode field, debounced 300ms.

```
GET /api/logistics/pincode/:code
x-store-id: <merchant_uuid>
```

Response decides: serviceable? COD-eligible? expected ETA? If not serviceable → block submit, show inline message, fire `pincode_blocked` analytics event.

Step 4 — Submit order (Gauthami)

```
POST /api/orders/direct
x-store-id: <merchant_uuid>
Content-Type: application/json

{
  "items": [{ "productId": "…", "variantId": "…", "qty": 1 }],
  "buyer":  { "name": "…", "phone": "…", "email": "…" },
  "shipTo": { "line1": "…", "city": "…", "state": "…", "pincode": "560001
  "payment": { "method": "COD" | "PREPAID", "intentId": "…?" }
}
```

The endpoint is **atomic**: stock decrement, order row, address row, and payment intent reservation succeed or roll back together.

Step 5 — Success response

```
{
  "ok": true,
  "data": {
    "orderId": "ord_…",
    "trackingId": "trk_…",
    "trackingUrl": "https://<merchant>.evoclabs.shop/track/trk_…"
  }
}
```

Storefront redirects to `/order/success?id=…` and renders the tracking link. **No email/auth gate** — `trackingUrl` is the customer's only handle.

Step 6 — Public tracking (Gauthami)

```
GET /api/orders/track/:trackingId
```

Returns coarse-grained status (`PLACED | PACKED | SHIPPED | OUT_FOR_DELIVERY | DELIVERED | CANCELLED`) plus partner timestamps. No buyer PII is returned beyond what the buyer entered.

---

5. Quality Standards

5.1 `safePrisma` is mandatory

All controller-layer Prisma calls **must** be wrapped in `safePrisma` (see `backend/src/config/database.js` and existing usages in `logisticsService.js`, `metaController.js`, `blogController.js`). This guards against:

- Connection pool exhaustion crashing the Node process.
- Unhandled `PrismaClientKnownRequestError` propagating to the user.
- Tenant-scope leaks when `where: { storeId }` is missing.

```
// pattern
const result = await safePrisma(
  () => prisma.product.findMany({ where: { storeId } }),
  { fallback: [], context: 'products.list' }
);
```

Code review will reject any direct `prisma.*` call in a request handler.

5.2 SEO-optimized metadata fetching

- Product pages **must** use Next.js `generateMetadata()` and consume `/api/products/:slug/meta`.
- Render `<title>`, `<meta name="description">`, OG, Twitter Card, and `application/ld+json` Product schema **server-side**.
- Honor merchant-defined `canonicalUrl` if present; otherwise default to the storefront URL.

- Cache headers from the API are authoritative — do not override `s-maxage` / `stale-while-revalidate`.

5.3 Real-time toast notifications

Every async user action must produce visible feedback within 200ms:

Event	Toast
Pincode valid	success — "Delivers to in 3–5 days"
Pincode invalid	error — "Sorry, we don't ship to this pincode yet"
Order placed	success — "Order placed! Track ID: trk_..." (with copy button)
Order failed	error — surface `error.message`, never raw stack
Stock changed mid-checkout	warning — "Only N left, quantity adjusted"

Use a single toast provider at the app root (`react-hot-toast` or equivalent). No `alert()`. No silent failures.

5.4 Operational guardrails

- **Rate limits:** Public endpoints (tracking, pincode, product reads) sit behind a per-IP bucket. SDKs should retry 429s with exponential backoff (max 3 attempts).
- **Idempotency:** `POST /api/orders/direct` accepts an `Idempotency-Key` header. The storefront **must** generate a UUID per checkout attempt and reuse it on retry.
- **PII discipline:** Buyer phone/email never enter analytics payloads, logs, or URL params. Tracking IDs are opaque and non-enumerable.
- **Observability:** Every SDK call propagates `meta.requestId` into client console errors so support can trace incidents.

---

6. Definition of Done

A merchant integration is shippable when:

1. Catalog SDK consumed by PDP and category pages, with typed responses and `safePrisma`-backed handlers.
2. Direct checkout flow works end-to-end on a real merchant tenant (header **and** subdomain modes).
3. Pincode validation blocks unserviceable orders before submit.
4. Order success page exposes a tokenized tracking URL accessible without auth.
5. Lighthouse SEO score ≥ 95 on PDP. Core Web Vitals "Good" on mobile.

6. All toasts are wired; no `console.error`-only failure paths remain.
7. Load test: 100 concurrent guest checkouts on a single tenant complete with < 1% error rate and no Prisma pool warnings.

Next actions

- Sameeha — scaffold `catalog-client` SDK, wire `generateMetadata()` on PDP.
- Gauthami — finalize `POST /api/orders/direct` contract, ship pincode endpoint behind feature flag.
- Joint — agree on shared `OrbitApiError` shape and idempotency-key conventions before SDK lock-in.